XINGQIANG CHEN

Hangzhou, China
+86 150-0133-4125 ◇ joy6677@outlook.com ◇ github.com/chenxingqiang

OBJECTIVE

Looking for **FULL TIME** roles in **AI Accelerator Compiler / System Software** · **LLM Inference Systems** · **HW/SW Co-design**.

INTERDISCIPLINARY BACKGROUND

Physics and biophysics training across computational chemistry, molecular dynamics, and computer science. Contests: **CUMCM National First Prize** (2012); **MCM Meritorious Winner** (2013); **Second Prize**, CCF Open Source GPU Innovation Ecosystem Regular Season S1, GPU Operator Optimization Challenge (2025).

EDUCATION

Doctoral Studies in Biophysics and Condensed Matter Sep. 2014 - Jul. 2018
Xiamen University, Department of Physics and Engineering; protein dynamics and QM/MM simulation
Certificate of completion; degree defense in progress

B.S. in Theoretical Physics Sep. 2010 - Jul. 2014
Xiamen University, Department of Physics and Engineering

SKILLS AND PROJECTS

Languages: C++, Python (primary); CUDA, Java, Scala, Go.

Compiler / Toolchain: MLIR, LLVM backends and ISA bring-up, TVM, graph partitioning, operator fusion, cost modeling, codegen, auto-tuning.

Chip Software / Verification: Palladium emulation, GDB co-debug against simulation models, firmware and weight-map generation, operator-level metrics (Cycles/MACs).

Inference Systems: Transformers, KV-Cache, quantization, batching and scheduling, Triton kernels, FlashAttention, CUPTI/Perfetto profiling.

Platforms: PyTorch, TensorFlow, ONNX, Kubernetes; Spark, Flink, Hadoop.

Open Source & Releases (`github.com/chenxingqiang` unless noted):

- **TraceSmith** — GPU profiling & replay across CUDA/ROCm/Metal/MACA/Ascend; C++ core, Python bindings, on PyPI. `TraceSmith`
- **YiRage Persistent Kernel** — LLM megakernel compiler and runtime, multi-backend. `YiRage`
- **JAX-sklearn** — sklearn-compatible API on JAX; 13,058 tests passing; on PyPI. `jax-sklearn`
- **Secret-Learn** — Privacy-preserving ML: federated learning, secret sharing, split learning. `Secret-Learn`
- **YouTube-DNN-RecSys** — TensorFlow + Spark retrieval and ranking (60 stars). `YouTube-DNN-RecSys`
- **Research CLI** — Command-line AI tooling for research workflows. `iechor-research/research-cli`
- **Questions-Gen** — Math-problem generation (Qwen3-14B, GRPO, distillation); CLI and HF weights. `GenQuestion`

EXPERIENCE

Compiler & Toolchain Expert, Witmem Technology
Hangzhou, China
Compiler / Palladium Verification *Dec. 10, 2025 - Present*

- Owned the product-line compiler and Palladium emulation for a CIM chip: model compile, codegen, on-chip execution, and defect isolation on the emulator.
- Brought up software on Palladium: ran compiler output on the emulator and jointly debugged with GDB across the simulation model, silicon, and system software.

- Implemented **CIM-MLIR**: PyTorch/TensorFlow/ONNX lowered via TOSA/StableHLO to CPU, GPU, and CIM, covering frontend, graph partition, hardware backend, and runtime.
- Implemented cost-driven graph partition and CIMHigh/Mid/Low IR; used OR-Tools for Die/L2/L0 allocation; emitted burnable **config.bin**, weight **map.bin**, GPU PTX, and LLVM host code.
- Implemented a cross-device runtime: per-device executors, unified memory, compute/transfer overlap and triple buffering; C++/Python APIs with latency, utilization, and transfer-cost counters.
- Implemented **GRC-PIM**: drift-gated, GEMV-reusing residual calibration and soft-decision error correction for PIM; submitted to *npj Unconventional Computing* (see Publications).

Senior AI Compiler Engineer, Yizhu Tech Ltd.
Hangzhou, China
R&D Department *Sep. 2022 - Dec. 2025*

- Led **YiRage Persistent Kernel (YPK)**, compiling end-to-end LLM decode into a single megakernel; **1.2×–6.7×** end-to-end latency gain over a PyTorch eager baseline on Llama2-7B, single A100 (varies with batch and sequence length).
- Implemented fused operators (RMSNorm+Linear, GQA) and a persistent execution model, cutting kernel-launch and intermediate-tensor round trips; largest gains on the batch=1 decode path.
- Built the scheduling layer and integrated Triton and FlashInfer kernels; multi-GPU scaling validated up to **8** devices with NVSHMEM-backed tensors.
- Developed graph compilation and auto-tuning: kernel parameter search and KV-cache-friendly memory layout to reduce memory traffic.
- Filed **2** invention patents; delivered the stack into **2** datacenter customer PoCs.
- Owned the LLVM backend Target and complex-ISA support for the in-house SoC, driving backend standardization and the software ecosystem.
- Built model-conversion and operator-level evaluation: integrated TVM, ONNX, MLCommons, TensorFlow/PyTorch into **YZlite**, reporting Cycles/MACs and multi-chip benchmarks. `ReRAM-Labs/yzlite`

Privacy-preserving Algorithms Engineer, Trustbe
Hangzhou, China
IT Group *Jan. 2022 - Sep. 2022*

- Built a homomorphic-encryption federated learning platform for cross-institution modeling in finance, improving model quality over local-only baselines while keeping privacy constraints.
- Designed MPC protocols for joint training and inference of bank risk models, balancing protocol security against communication cost.
- Open-sourced **Secret-Learn**: sklearn-compatible API across federated learning, secret sharing, and split learning, JAX-accelerated and integrated with SecretFlow; on PyPI. `Secret-Learn`
- Filed **3** invention patents, all published: `CN114925786A`, `CN114997345A`, `CN115130681A`.

Staff Algorithms Engineer, BioMap
Beijing, China
Algorithm Engineering Group *Sep. 2021 - Dec. 2021*

- Built bioinformatics tooling and a unified data platform for large-scale sequence analysis, structure prediction, and functional annotation.
- Set up evaluation pipelines for protein structure prediction models (AlphaFold family) feeding downstream drug design.
- Two registered software copyrights (2021SR2004464, 2021SR1862397); open-sourced AlphaFold2 code walkthroughs (26 stars). `alphafold2-codecs`

Senior Algorithms Engineer, Yalla Group (NYSE: YALA)
Hangzhou, China
Data Center Engineering Group *Sep. 2020 - Sep. 2021*

- Rebuilt the ChatRooms recommender with multi-objective ranking; A/B tests showed significant gains in daily active time and 7-day retention (**42% → 61%**).
- Built Ludo risk control from real-time behavior features and ML models, materially reducing fraudulent refunds.
- Developed a card-game AI with deep RL for human-vs-bot play and matchmaking fill, improving off-peak match quality and retention.

Senior Algorithms Engineer, Gejia Network Tech — Hangzhou, China
Search & Recommendation Engineering Group — *Mar. 2019 - Sep. 2020*

- Designed multi-scenario ranking with collaborative filtering, deep ranking models, and a product knowledge graph; online experiments delivered double-digit relative CTR and CVR gains.
- Optimized the serving path with feature caching and candidate pruning, holding P99 latency under **50ms** through peak promotional traffic.
- Open-sourced YouTube-DNN-RecSys, a TensorFlow + Spark retrieval and ranking reference (60 stars). `chenxingqiang/YouTube-DNN-RecSys`

Machine Learning Engineer, Bianlifeng Tech — Beijing, China
Data Engineering Group — *Mar. 2017 - Mar. 2019*

- Built store-level sales forecasting across **50+** stores using gridded weather and spatiotemporal features, reducing stock-outs and dead stock while improving turnover.
- Linked customer identities to purchase history over tens of millions of events to power personalized recommendations.
- Shipped a product recognition model for **500+** SKUs used in fast checkout, cutting checkout and queue time.

RESEARCH & INTERNSHIPS

Molecular Dynamics Simulation of Proteins with QM/MM model — Xiamen university, China
(China National Funds) — *Sep. 2014 - Mar. 2017*

- Ran QM/MM protein dynamics simulations and conformational analysis; results published in Acta Cryst. D, PLoS ONE, and Phys. Rev. E (see Publications).

Liquid Biopsy ct-DNA Kit and DNA Methylation Analysis — Lepu Medical, China
(R&D Engineer Intern) — *Mar. 2016 - Mar. 2017*

- Analyzed roughly 10,000 public gene-chip samples (TCGA/GEO) to localize Septin9 methylation sites, an early colon-cancer marker, informing biomarker selection for a clinical diagnostic kit.

AWARDS & SERVICE

- **National First Prize**, China Undergraduate Mathematical Contest in Modeling (CUMCM), 2012.
- **Meritorious Winner**, Mathematical Contest in Modeling (MCM/COMAP), 2013.
- **6th** worldwide, International Solar Decathlon Competition, 2013.
- Ranks **57** and **32** of 1739 teams, IEEE ICDM Global Meteorological AI Challenge, 2018.
- **Second Prize**, 8th CCF Open Source Innovation Competition — Open Source GPU Innovation Ecosystem Regular Season S1, GPU Operator Optimization Challenge (CCF/ODTC), 2025. CCF announcement
- Judge, first screening, Best Practices of Applied Algorithms, 2021.

PUBLICATIONS

- Chen X. GRC-PIM: A Drift-Gated, GEMV-Reusing Residual Calibrator for Soft-Decision Error Correction in Processing-in-Memory. *npj Unconventional Computing* (submitted).
- Cai Y, Su M, Ahmad A, Hu X, Sang J, Kong L, Chen X, Wang C, Shuai J, Han A. Conformational dynamics of the essential sensor histidine kinase WalK. *Acta Crystallographica Section D: Structural Biology.* 2017 Oct 1;73(10):793-803.
- Xia H, Chen X, Chen X, Cheng J, Liu Y, Chen X, Zhang Q, Zou G. Chiral discrimination of amino acid enantiomers based on different interactions with Cu2+. *Sensors and Actuators B: Chemical.* 2018 Jan 1;254:44-51.
- Ahmad A, Cai Y, Chen X, Shuai J, Han A. Conformational dynamics of response regulator RegX3 from Mycobacterium tuberculosis. *PloS one.* 2015 Jul 22;10(7):e0133389.
- Chen Y, Qi H, Li X, Cai M, Chen X, Liu W, Shuai J. Suppressing effect of C a 2+ blips on puff amplitudes by inhibiting channels to prevent recovery. *Physical Review E.* 2016 Aug 15;94(2):022411.

Open Source Architecture Appendix

Chen Xingqiang · github.com/chenxingqiang · draw.io source in docs/architecture/drawio/

TraceSmith

GPU Profiling & Replay
CUPTI / ROCm / Metal / MACA / Ascend

YiRage YPK

LLM megakernel compiler
12 backends + RuntimeFusion

JAX-sklearn

sklearn API + JAX proxy
CPU / GPU / TPU fallback

Secret-Learn

191 algos × FL / SL / SS
JAX + SecretFlow

YouTube-DNN

Two-tower recall
TensorFlow + Spark

Research CLI

AI CLI + academic tools
multi-provider / MCP

Questions-Gen

Qwen3-14B contest problems
Pretrain → GRPO → Distill

Each following page is a draw.io diagram exported to SVG, then appended to the resume PDF.

TraceSmith — Cross-platform GPU Profiling & Replay

github.com/chenxingqiang/TraceSmith · C++ core + pybind11 · PyPI: tracesmith

1. Workloads / producers

PyTorch / Kineto
kernel + runtime traces

TensorFlow / XLA
graph + device events

hooks

Custom CUDA / HIP
driver / runtime APIs

Multi-GPU cluster
NCCL collectives

2. User interface

CLI
profile / record / view
export / analyze / replay

Python API
pybind11 bindings
in-process capture

Distribution
pip install tracesmith
Apache-2.0

3. Capture backends (platform-specific, lock-free ingest)

NVIDIA CUPTI
nsys / kernel / memcpy

AMD ROCm
rocprof / HIP events

Apple Metal
xctrace / Instruments

MetaX MACA
MCPTI

Huawei Ascend
CANN / msprof

Linux eBPF
host syscall / driver

events

4. Core pipeline

SPSC ring buffer
lock-free ingest
<1% overhead

Common / stacks
event timeline
XRay / host stacks

export

SBT format
string interning
delta timestamps

State / timeline
GPU state machine
stream reconstruction

5. Replay, export, debug

Replay engine
full / partial / dry-run
stream scheduler
determinism checker

Perfetto
protobuf export
~85% smaller than JSON

Tracy profiler
live timeline view

GDB RSP
GPU breakpoints
trace-driven debug

Cluster sync
NTP / PTP / CUDA
NCCL tracking

Flow: workload → CLI/Python → platform backend → lock-free ring → SBT/timeline → replay or Perfetto/Tracy/GDB.

YiRage (YPK) — Multi-backend LLM Megakernel Compiler & Runtime

github.com/chenxingqiang/YiRage · based on Mirage (CMU) + multi-backend / RuntimeFusion extensions

1. Serving / embedding

vLLM
keeps schedule + KV
RF selects capsules

SGLang
decode-loop-erdecode
same FusionPlan cache

Python API
PersistentKernel
HardwareRegistry

UnifiedCompiler
graph + backend pick
Cython CoreBridge

2. RuntimeFusion

FusionPlan cache
search / profile results
per backend + shape

FusionCapsule
one decode-step kernel
selected each token

Device memory
persistent workspace
KV-friendly layout

cache

3. Compiler / search

Kernel graph IR
matmul / attn / norm
mark inputs / outputs

Superoptimize
hardware-aware search
15 candidate dims

Fingerprint
numeric verify
before accept

Autotune
grid / tile / pipeline
occupancy model

4. Threadblock operators + transpilers

MatMul
Tensor Core / Cube

Attention / GQA
Flash-style tiles

RMSNorm / SwiGLU
fused decode path

Triton codegen

NKI / MLIR

CUDA / HIP JIT

5. Backend registry (12 targets; factory + thread-safe singleton)

CUDA
Tensor Core
bank conflict

ROCm
HIP / MFMA

MPS
Apple Metal
unified mem

Ascend
AI Core / Cube
BiSheng

MACA
MetaX

TPU / XPU
NKI / Intel

MLIR / CPU
LLVM / SIMD

FPGA
experimental

Serving keeps scheduling/KV; compiler searches fused kernels; RuntimeFusion injects FusionCapsules into vLLM/SGLang decode.

JAX-sklearn (xlearn) — Drop-in sklearn with automatic JAX acceleration

github.com/chenxingqiang/jax-sklearn · 13,058 tests passed · pip install jax-sklearn

1. User code (zero API change)

import xlearn as sklearn
LinearRegression().fit(X, y)

Same estimators
cluster / decompose / preprocess

config_context()
force GPU / disable JAX

2. sklearn-compatible surface

Linear
LR / Ridge / Lasso
ElasticNet

Cluster
KMeans
282 tests

Decompose
PCA / TruncatedSVD
528 tests

Preprocess
Standard / MinMax

Others
native sklearn
via fallback

3. Runtime injection / proxy

__getattr__ lazy import
xlearn.linear_model …

JAX

_auto_jax_accelerate
replace estimator classes

EstimatorProxy
create JAX or original impl

Heuristics
fallback3 — KMeans > 1e6
PCA > 1e7 complexity

4. Acceleration + data

JAX kernels
JIT / vmap
universal mixins

Algo kernels
GP / MLP / SVM / tree

NumPy ↔ JAX
dtype / shape guards

Fallback
small data or error
→ original sklearn

5. Hardware abstraction (auto-select by problem size)

CPU
<10K samples
1.0–2.5×

NVIDIA GPU
10K–100K
5.5–8×

Google TPU
>100K
9.5–15×

Apple Metal
M1/M2/M3 beta
2–4×

Proxy decides JAX vs sklearn from dataset size, algorithm family, and configured platform; errors fall back silently.

Secret-Learn — Privacy-preserving ML (191 algorithms × FL / SL / SS)

github.com/chenxingqiang/Secret-Learn · sklearn API · JAX acceleration · SecretFlow devices

1. Applications

- **Healthcare** — hospitals keep records local
- **Finance** — cross-bank fraud / risk
- **IoT / edge** — federated sensors
- **Research** — multi-institution studies

2. sklearn-compatible API (191 estimators, 30+ categories)

- Linear / GLM
- Ensemble / trees
- SVM / neighbors
- Cluster / PCA
- NN / Naive Bayes
- Preprocess / select

3. Privacy mode router

- **FL — Federated**
  data stays on PYU
  ~~local JAX train~~
  HEU secure aggregate
  best: horizontal FL
- **SL — Split learning**
  model split by party
  ~~local JAX~~
  encrypted activations
  vertical / deep models
- **SS — Secret sharing**
  SPU MPC encrypt
  maximum privacy
  multi-party runtime

4. Intelligent generation + acceleration

- **Algorithm classifier**
  supervised? partial_fit?
  fit(x) vs fit(x,y)
- **Template migrator**
  auto-generate FL/SL/SS
  adapter from sklearn
- **JAX-sklearn core**
  local 5×+ speedup
  CPU / GPU / TPU
- **573 examples**
  one per algo × mode
  batch runners

5. SecretFlow device plane

- **PYU**
  plaintext local
  party compute
- **HEU**
  homomorphic
  secure aggregate
- **SPU**
  MPC / secret share
- **TEE**
  trusted execution
- **DP hooks**
  optional noise

Same estimator call; mode picks FL/SL/SS. FL keeps data local, SL splits the net, SS runs under MPC on SPU.

YouTube-DNN-RecSys — Two-tower recall (Covington et al.) in TF + Spark

github.com/chenxingqiang/YouTube-DNN-RecSys · Python TensorFlow · Scala Spark · 60 GitHub stars

1. Event / feature sources

Watch history
video / search ids

Context
device / geo / time

Continuous feats
age / counts

Candidate items
catalog embeddings

2. Dual implementation stacks

Python / TensorFlow
data2tfrecords.py
models/dnn.py (Estimator)
custom layers / TensorBoard
SavedModel export

Scala / Spark
FeatureBuilder.scala
BaseSparkLocal
PredictUserVector
ItemEmbeddingPredictor

3. Two-tower model

User tower
item-id embeddings
average watch history
concat continuous feats
MLP → user vector u

Matching
dot(u, v_i)
sampled softmax
next-watch objective

Item tower
shared item embedding
table v_i
optional side features

4. Train / serve

Train
TF Estimator
optional PS / workers
checkpoints

Eval
hold-out next-item
TensorBoard curves

Online recall
ANN / softmax top-k
user vec × item table

Artifacts
TFRecords
SavedModel / Spark vecs

History embeddings are mean-pooled into a user vector and scored against the item table — the paper's candidate-generation stage.

Research CLI — Command-line AI workflow with academic research tools

github.com/iechor-research/research-cli · TypeScript monorepo · Gemini CLI lineage + multi-provider + research modules

1. Interaction

packages/cli
Ink TUI · research.tsx

Slash commands
/research /paper /submit
/model config|select

Non-interactive
npx / CI scripts

VS Code companion
packages/vscode-ide

2. packages/core — orchestration

Prompt + context
GEMINI.md
file discovery

Session state
tool calls ~~compress~~
1M-token window

Policy / sandbox
~~tool permission~~
workspace isolation

Routing
provider + model
fallback chain

3. Multi-model providers

Gemini / Vertex

OpenAI

Anthropic

Baidu ERNIE

DeepSeek

Qwen / Groq / Kimi

4. Tools + MCP

Workspace tools
fs / shell / git

MCP servers
external capabilities

Search grounding
iEchor / web

SDK / a2a-server
programmatic embed

5. Academic research modules (packages/core/src/tools/research)

Bibliography
arXiv / PubMed / IEEE
BibTeX export

Writing
outline / grammar
LaTeX manager

Analysis
stats / citation net
experiment codegen

Submission
journal matcher
template / package

Prompt → core context → selected provider → tools/MCP/research modules → streamed terminal result.

Questions-Gen — Math competition problem generator (Qwen3-14B)

github.com/chenxingqiang/GenQuestion · Unsloth training · Hugging Face: xingqiang/* · pip install questions-gen

1. Training mixture

Historical contests
50% · CUMCM / olympiad

Conditional variants
30% · rewrite constraints

Innovative types
20% · new stems

data_preparer.py
tokenize / pack

2. Three-stage pipeline

Stage 1 — Pretrain
Qwen3-14B SFT
HF: stage1-fp-merged
base generation

Stage 2 — GRPO
8 problems / group
novelty constraint
policy optimization

score

Reward
0.4 difficulty
0.3 novelty
0.2 rigor
0.1 diversity

eval

Stage 3 — Distill
DeepSeek-R1 teacher
difficulty / logic / edu
final merged FP16

3. Validation (questions_gen/validation)

ModelValidator
single-model n tests

BatchValidator
algebra / calculus
geometry / number theory

QualityEvaluator
content / clarity
difficulty / originality

Scores
Final 0.847
teacher 4.2 / 5

4. Package / deploy

CLI
questions-gen
validate / batch
quality / compare

Python API
QuestionsGenTrainer
OllamaManager
HuggingFaceUtils

Hugging Face
stage1 / stage2 / final
xingqiang/*

Ollama
push-all
local generate API

Pretrain → GRPO with multi-dimensional rewards → R1 distillation → validate by category → publish HF weights and Ollama models.